

MICROSOFT SQL SERVER & AZURE SQL

DP-800 Study Guide

Developing AI-Enabled Database Solutions

A concise revision guide for SQL AI developers

Prepared by Jitendra Singh Malik

September 2026

Independent study resource

These notes were created while preparing for the DP-800 exam. They are not official Microsoft exam material. Features, previews, licensing and user-interface details may change, so validate current behaviour using Microsoft Learn and product documentation.

How to use this guide

This guide condenses DP-800's three skills-measured domains into a structured reference, grounded in Microsoft's official study guide and SQL Server 2025 / Azure SQL documentation.

- Read chapters 1-3 in order to build up from schema design and T-SQL, through security and deployment, to native AI capabilities.
- Use the code-identifier callouts (in monospace) to memorize exact T-SQL syntax, function names, and configuration options.
- Review the "Common confusion" callouts before practice exams - they target the exact distinctions exam scenarios test.
- Use the final quick-reference chapter as a last-day revision sheet.

Contents

1. Design and Develop Database Solutions (35-40%)
2. Secure, Optimize, and Deploy Database Solutions (35-40%)
3. Implement AI Capabilities in Database Solutions (25-30%)

Source note

This guide is based on FabricPrep's DP-800 deep-dive study guide pages, researched directly from Microsoft Learn's official DP-800 study guide and SQL Server 2025 / Azure SQL documentation.

CHAPTER 1

Design and Develop Database Solutions

Build tables, indexes, and specialized object types; write advanced T-SQL; and safely bring GitHub Copilot and MCP into the database development workflow.

Table design: data types, indexes, and specialized table types

Table design is the foundation DP-800 builds everything else on top of — the exam expects you to know not just `CREATE TABLE` basics, but which of SQL Server's several specialized table types fits a given scenario, since picking the wrong one is a common trap in scenario questions.

Constraints and identity patterns

- `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `CHECK`, and `DEFAULT` constraints are enforced by the engine at write time, which is why they're preferred over application-level validation for anything that must never be violated regardless of which client writes the row.
- `SEQUENCE` objects generate numeric values independently of any single table — unlike an `IDENTITY` column, a sequence can be shared across multiple tables, pre-fetched in application code with `NEXT VALUE FOR`, and its increment/cycle behavior configured after creation, which `IDENTITY` doesn't allow.

Columnstore indexes and JSON columns

- A columnstore index stores data column-by-column rather than row-by-row, giving large analytical scans dramatically better compression and I/O than a traditional rowstore index — the trade-off is that it's optimized for bulk scan/aggregate patterns, not high-frequency single-row lookups or updates.
- The native JSON data type (SQL Server 2025, Azure SQL Database, Azure SQL Managed Instance) stores documents in an optimized binary format instead of plain `nvarchar`, giving faster reads/writes and smaller storage than string-based JSON, with no application code changes needed since the same `JSON_VALUE/JSON_QUERY/OPENJSON` functions work against it.
- A `CREATE JSON INDEX` (SQL Server 2025 preview) accelerates `JSON_VALUE` equality/range predicates and the `JSON_CONTAINS` function against a `json` column, the same way a regular index accelerates a `WHERE` clause on a relational column.

Specialized table types

- **Temporal tables** (`PERIOD FOR SYSTEM_TIME`, `SYSTEM_VERSIONING = ON`) automatically keep a full history of every row change in a linked history table, letting you query `FOR SYSTEM_TIME AS OF` a past point without any application-level auditing code.
- **In-memory tables** (`MEMORY_OPTIMIZED = ON`) live primarily in memory with optionally durable logging, built for workloads with extreme concurrent insert/update rates where traditional locking becomes the bottleneck.
- **External tables** point at data living outside the database (another SQL instance, or a data lake via PolyBase-style connectivity) so it can be queried with normal T-SQL without first importing it.
- **Ledger tables** (`LEDGER = ON`) add cryptographically verifiable, tamper-evident history — `APPEND_ONLY = ON` blocks `UPDATE/DELETE` entirely at the API level for insert-only audit patterns, while an updatable ledger table (the default once `LEDGER = ON` is set) still allows updates and deletes but chains every change into a verifiable digest.
- **Graph tables** (`AS NODE / AS EDGE`) model many-to-many relationships natively — covered in depth alongside the `MATCH` operator later in this topic.

Partitioning

- Table and index partitioning splits data across multiple physical units (a partition function plus a partition scheme) based on a partitioning column, most commonly a date — this speeds up maintenance operations that only touch recent data (like purging old rows via SWITCH) and can improve query performance when queries filter on the partitioning column, at the cost of added schema complexity.

Common confusion

Ledger tables and temporal tables both keep history, but for different reasons: temporal tables exist so *any* authorized user or application can query past states for business logic (an as-of report), while ledger tables exist specifically to make tampering by a privileged user (a DBA editing rows directly) cryptographically detectable — a ledger table can also be temporal under the hood, but the two features solve different trust problems and aren't interchangeable.

Programmability objects: views, functions, procedures, and triggers

Programmability objects are how you package T-SQL logic into a reusable, permission-able unit — the exam tests whether you know which object type fits a given need, since all four can sometimes achieve a similar-looking result.

Views

- A view is a saved SELECT statement queried like a table — useful for hiding join complexity from consumers, presenting a restricted column set without granting table-level access, or maintaining a stable interface while the underlying schema evolves.
- WITH SCHEMABINDING locks the view's definition to its base tables' current schema, preventing a column the view depends on from being dropped or changed underneath it — required if the view will itself be indexed.

Scalar and table-valued functions

- A scalar function returns a single value and can be called inline in a SELECT list or WHERE clause, but a scalar function that isn't inlineable by the optimizer can silently force row-by-row execution across a large result set — a well-known performance trap.
- An inline table-valued function (iTVF) returns a table from a single RETURN (SELECT ...) statement and is inlined into the calling query's plan, making it the generally preferred pattern over a scalar function when a per-row computation can be reshaped as a set-based one — this is exactly the technique used to write a row-level security predicate function.
- A multi-statement table-valued function builds its result set with multiple statements into a declared @table variable — more flexible, but not inlined by the optimizer the way an iTVF is.

Stored procedures

- Stored procedures accept input/output parameters, can contain multiple statements and control-of-flow logic, and are the natural home for a unit of work an application calls as a single round trip (an order-placement transaction, a batch update) rather than a value the optimizer needs to reason about inline.
- Because a stored procedure's plan is cached and reused, parameterized procedures avoid the ad hoc query plan-cache bloat that comes from sending the same shape of query as a literal string over and over.

Triggers

- AFTER (or FOR) triggers fire once the triggering INSERT/UPDATE/DELETE has completed, and are the classic mechanism for cascading side effects (an audit log write, a denormalized summary update) that must happen atomically with the original change.

- `INSTEAD OF` triggers replace the triggering statement entirely — the standard technique for making an otherwise non-updatable view (one spanning multiple base tables) accept `INSERT/UPDATE/DELETE` statements by translating them into operations against the correct underlying tables.

Common confusion

A view and an inline table-valued function look similar (both wrap a `SELECT` and both get inlined into the caller's plan), but only a function can take parameters — a view's filtering logic has to live in the caller's `WHERE` clause, while a function can encapsulate parameterized logic (like a security predicate that takes a user ID) that a view structurally can't express on its own.

Modern T-SQL: CTEs, window functions, correlated queries, and error handling

Beyond basic joins and aggregates, DP-800 expects fluency with the T-SQL constructs that solve problems a plain `GROUP BY` can't — hierarchical data, running calculations, and row-by-row comparisons against related rows.

Common table expressions

- A CTE (`WITH cte_name AS ( ... )`) is a named, scoped result set that can be referenced multiple times in the statement that follows it — mainly a readability tool for breaking a complex query into named steps, functionally similar to a derived subquery.
- A *recursive* CTE (a base case `UNION ALL`'d with a recursive member referencing the CTE itself) is the standard way to walk a hierarchy — an org chart, a bill-of-materials tree, or a category tree with arbitrary depth — without knowing the depth in advance.

Window functions

- Window functions (`OVER (PARTITION BY ... ORDER BY ...)`) compute a value across a set of rows related to the current row *without* collapsing them into one row the way `GROUP BY` does — a running total, a rank within a group, or a row's value compared to the previous row all stay at the original row grain.
- `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()` differ specifically in how they handle ties: `ROW_NUMBER()` always assigns unique sequential numbers even to tied rows, `RANK()` gives tied rows the same rank and then skips the next rank(s), and `DENSE_RANK()` gives tied rows the same rank with no gap afterward.
- `LAG()`/`LEAD()` read a value from a preceding/following row in the same result set (commonly used for period-over-period comparisons), while framing clauses like `ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW` control exactly which rows a running aggregate accumulates over.

Correlated subqueries

- A correlated subquery references a column from the outer query, so it's logically re-evaluated once per outer row (find each customer's most recent order, or each product's price versus its category average) — semantically distinct from an uncorrelated subquery, which is evaluated once regardless of the outer row.
- Many correlated subqueries can be rewritten as a join or a window function, and the optimizer often produces a better plan from the rewritten form — worth checking when a correlated subquery shows up as a performance bottleneck in a query plan.

Error handling

- `TRY...CATCH` wraps a block of T-SQL so that a runtime error transfers control to the `CATCH` block instead of aborting the batch outright — `ERROR_NUMBER()`, `ERROR_MESSAGE()`, `ERROR_LINE()`, and `ERROR_PROCEDURE()` inside the `CATCH` block describe exactly what failed.
- Inside a transaction, a `CATCH` block typically checks `XACT_STATE()` before deciding to `COMMIT` or `ROLLBACK` — a value of `-1` means the transaction is in an uncommittable state and must be rolled back, while `1` means it can still be committed or rolled back at the caller's discretion.

Common confusion

A window function and a `GROUP BY` aggregate can compute the same underlying math (a sum, a count) but return fundamentally different shapes: `GROUP BY` collapses the result to one row per group, while a window function keeps every original row and attaches the computed value alongside it — needing both a detail row and an aggregate value side by side is exactly when a window function is the right tool and `GROUP BY` alone isn't.

Pattern matching at scale: JSON, regular expressions, fuzzy matching, and graph queries

SQL Server 2025 (and the equivalent Azure SQL / Fabric SQL database surfaces) added a cluster of pattern-matching functions the exam calls out by name — these let you do text and structural matching natively in T-SQL instead of round-tripping data to application code.

JSON construction and search functions

- `JSON_OBJECT('name':value, ...)` and `JSON_ARRAY(value, ...)` build JSON text directly from SQL expressions, and `JSON_OBJECTAGG/JSON_ARRAYAGG` do the aggregate equivalent — turning a whole result set's rows into one JSON object or array, which is exactly the shape an LLM prompt or a REST response usually needs.
- `OPENJSON(json_expression [, path]) WITH (...)` is still the standard way to shred a JSON document back into relational rows and columns, and `JSON_VALUE/JSON_QUERY` still extract a scalar or an object/array from JSON text respectively.
- `JSON_CONTAINS(target, search_value [, path])` (SQL Server 2025 preview) tests whether a value, object, or array is present anywhere inside a JSON document or at a specific path, returning `1/0/NULL` — a purpose-built alternative to writing an `OPENJSON` cross-apply just to check for containment, and one that a JSON index can accelerate.

Regular expression functions

- `REGEXP_LIKE(string_expression, pattern [, flags])` returns a boolean and is the direct regex replacement for a `LIKE` predicate that needs more power than `%` and `_` wildcards provide — it requires database compatibility level 170 or higher.
- `REGEXP_REPLACE`, `REGEXP_SUBSTR`, `REGEXP_INSTR`, and `REGEXP_COUNT` mirror familiar regex operations (replace, extract, locate, count matches); `REGEXP_MATCHES` and `REGEXP_SPLIT_TO_TABLE` are table-valued, returning one row per captured match or per split segment respectively.
- Flags (`i` case-insensitive, `m` multi-line, `s` dot-matches-newline, `c` case-sensitive/default) are passed as a short string argument, and `REGEXP_REPLACE` also lets you target the *n*th occurrence rather than replacing every match.

- Regex-based dynamic data masking (`MASKED WITH (FUNCTION = 'REGEXP_REPLACE("<pattern>", "<replacement>")')`, Azure SQL Database preview) applies the same pattern-matching engine to obscure part of a column's value — for example masking all but a phone number's country code — rather than only the coarser `default()/partial()/email()/random()` masks.

Fuzzy string matching

- `EDIT_DISTANCE(str1, str2 [, maximum_distance])` implements Damerau-Levenshtein distance — the number of insertions, deletions, and substitutions needed to turn one string into another — and `EDIT_DISTANCE_SIMILARITY` expresses the same comparison as a normalized 0-100 score instead of a raw count.
- `JARO_WINKLER_DISTANCE/JARO_WINKLER_SIMILARITY` implement a different algorithm that gives extra weight to strings that match from the beginning — generally a better fit than edit distance for things like matching names or product codes where a shared prefix should count for more.
- These functions are preview features that require enabling `PREVIEW_FEATURES` via `ALTER DATABASE SCOPED CONFIGURATION`, and only support Windows or binary (`BIN/BIN2`) collations — a database using a non-binary `SQL_*` collation needs an explicit `COLLATE` clause on the arguments before the functions will accept them.

Graph queries with MATCH

- `CREATE TABLE ... AS NODE` and `AS EDGE` create graph tables; an edge constraint (`CONSTRAINT ... CONNECTION (NodeA TO NodeB)`) restricts which node types an edge can legally connect, the graph equivalent of a foreign key.
- The `MATCH` clause uses ASCII-art syntax to express a traversal — `MATCH(Person1-(friendOf)->Person2)` finds direct connections, and chaining the pattern `(Person1-(friendOf)->Person2-(friendOf)->Person3)` finds friends-of-friends without a self-join for every hop.
- `SHORTEST_PATH` (with a quantifier like `{1, 3}`) finds the shortest route between nodes across a variable number of hops — the tool for "how are these two entities connected" questions that a fixed-depth `MATCH` pattern can't answer.

Common confusion

`REGEXP_LIKE` and the classic `LIKE` operator both test a string against a pattern, but `LIKE`'s wildcard vocabulary (`%, _, [ . . . ]`) is far more limited than full regular expression syntax (alternation, quantifiers, character classes, capture groups) — reaching for `REGEXP_LIKE` only when `LIKE` genuinely can't express the pattern keeps queries both correct and easier to optimize, since a plain `LIKE` with a leading literal can still use an index seek in ways a regex generally can't.

AI-assisted SQL development: GitHub Copilot, Copilot in Fabric, and MCP

DP-800 is the first Microsoft database exam to test AI-assisted *development* workflow directly — not just AI features inside the database, but how you safely bring an AI coding assistant into the SQL authoring process itself.

Enabling and configuring Copilot for SQL work

- GitHub Copilot integrates into SQL tooling (SSMS's AI Assistance workload, the MSSQL extension for VS Code, and Copilot in Microsoft Fabric) to generate T-SQL, explain existing queries, and suggest schema changes from natural-language prompts — Agent mode specifically is what's required to use MCP tools, since plain Ask mode doesn't support them.

- Model and tool options are configured per Copilot chat session — which underlying model answers a prompt, and which MCP tools (if any) are enabled for that session — separate from whether Copilot itself is licensed and turned on for the organization.
- An organization's GitHub Copilot administrator can turn off Agent mode/MCP access tenant-wide, and can maintain an MCP server allow list restricting exactly which servers are permitted to connect — a governance layer worth knowing exists independently of any individual developer's local configuration.

GitHub Copilot instruction files

- An instruction file (a `.github/copilot-instructions.md`-style file checked into the repository) gives Copilot durable, project-specific context — coding conventions, naming standards, which schema patterns to prefer — so every suggestion in that repository follows the same guidance without the developer re-explaining it in every prompt.
- Because instruction files are just files under source control, they go through the same pull-request review as any other code change, which is what makes "the AI assistant's standing instructions for this project" itself an auditable, versioned artifact rather than personal, undocumented tribal knowledge.

Connecting to MCP server endpoints

- The Model Context Protocol (MCP) is the open standard that lets an AI coding assistant call external tools through a client-server model: the assistant (an MCP *client*) requests actions, and an MCP *server* exposes a well-defined set of tools it's allowed to perform.
- A SQL Server or Azure SQL MCP server exposes tools like listing schemas/tables/views/functions and running an approved query, letting Copilot answer "what tables are in this database" or generate and execute T-SQL directly against live schema context instead of guessing at column names.
- A Fabric lakehouse or Fabric Data Warehouse MCP server does the analogous thing for Fabric items — the SQL analytics endpoint's server, for example, exposes a single `executeSQL` tool that runs an approved T-SQL statement and returns the result, deliberately not exposing a separate metadata-browsing tool (schema questions are instead answered by having the agent query `INFORMATION_SCHEMA` through that same tool).
- MCP tools are disabled by default the moment a server is added — each tool has to be explicitly enabled in the Tools list before Copilot can invoke it, and a destructive or sensitive tool call still prompts for explicit approval before it runs.

Security implications of AI-assisted tooling

- Because an MCP-connected Copilot session can execute real T-SQL against a real database, the account or connection profile it uses should carry the least privilege that still lets it do its job — an assistant wired up with elevated (say, `db_owner`) credentials turns a mistaken or misinterpreted natural-language prompt into a real schema or data change.
- Reviewing AI-suggested T-SQL before running it matters more, not less, once MCP is involved, since a suggestion is no longer just inert text in a chat window — approving a tool call is the point where a hallucinated or overly broad statement (an unfiltered `DELETE`, an unintended schema change) actually executes.

Common confusion

Enabling GitHub Copilot for a SQL project and connecting it to an MCP server are two separate steps that are easy to conflate: Copilot alone can generate T-SQL text based on what's visible in your editor and instruction files, but it can't see live database schema or execute anything until an MCP server is added, its tools are individually enabled, and (in Agent mode) each tool call is approved.

CHAPTER 2

Secure, Optimize, and Deploy Database Solutions

Lock down data with encryption, masking, and row-level security; tune performance with execution plans and Query Store; and ship schema changes through SQL Database Projects and Data API builder.

Data security: encryption, masking, row-level security, and passwordless access

Data security on DP-800 spans several independent, layered controls — the exam expects you to know which layer stops which threat, since a database can pass one control and still be wide open through another.

Always Encrypted and column-level encryption

- Always Encrypted protects sensitive columns *in use*, not just at rest or in transit: encryption and decryption happen client-side inside a driver that holds the column encryption key, so plaintext values never reach the SQL Server or Azure SQL engine itself — even a DBA with full sysadmin rights only ever sees ciphertext when querying the column directly.
- *Deterministic* encryption always produces the same ciphertext for the same plaintext value, which allows equality comparisons, joins, and grouping on the encrypted column — at the cost of being vulnerable to frequency analysis on low-cardinality data (an attacker who can see ciphertext patterns can infer which encrypted values repeat most often).
- *Randomized* encryption produces different ciphertext every time, which is more secure but disallows searching, joining, or grouping on that column entirely — the classic trade-off is deterministic for columns you must query on (a national ID used as a lookup key) and randomized for columns you only ever retrieve (a stored secret answer).
- Always Encrypted *with secure enclaves* relaxes that trade-off by performing richer operations (pattern matching, range comparisons, in-place cryptographic re-keying) inside a hardware- or software-based trusted execution environment, without plaintext ever being visible to the database engine outside the enclave.

Dynamic Data Masking

- `ALTER TABLE ... ALTER COLUMN col ADD MASKED WITH (FUNCTION = 'default()')` (or `email()`, `partial(prefix, [padding], suffix)`, `random(low, high)`) obscures a column's *displayed* value for users without the UNMASK permission, while the real value is stored unmasked and unchanged in the table.
- Because masking is a display-time transformation rather than actual encryption, it's meant to reduce accidental over-exposure to a broad set of ordinary users (support staff running ad hoc queries) — it is explicitly not a substitute for encryption or access control against a determined or privileged attacker, since the underlying data is fully readable to anyone granted UNMASK or with sufficient permission to query it another way.

Row-Level Security

- Row-Level Security is implemented as an inline table-valued function (a *security predicate*) bound to a target table through `CREATE SECURITY POLICY ... ADD FILTER PREDICATE fn(column) ON schema.table`; a `FILTER` predicate silently hides non-matching rows from `SELECT/UPDATE/DELETE`, while a `BLOCK` predicate actively rejects a write (`AFTER INSERT`, `AFTER UPDATE`, `BEFORE UPDATE`, or `BEFORE DELETE`) that would violate it.
- Because the predicate function is just T-SQL, it commonly compares a row's tenant/owner column against `USER_NAME()`, `SESSION_CONTEXT()` (a value the application sets after connecting with a shared, lower-privileged login), or a role membership check — the policy is what wires that comparison to a specific table and enforces it automatically on every query, so application code never has to remember to add the filter itself.
- A security policy created with the default `SCHEMABINDING = ON` locks the predicate function and any tables/functions it references, and bypasses extra permission checks on those dependencies when a user queries the protected table — turning it `OFF` requires granting those permissions separately.

Object-level permissions and passwordless access

- Standard `GRANT/DENY/REVOKE` on schemas, tables, views, and procedures remains the first line of access control underneath any of the row- or column-level features above — RLS and masking narrow what's visible *within* an object a principal can already access, they don't substitute for object-level grants.
- Passwordless, Microsoft Entra ID–based authentication (a managed identity or a federated user) removes a stored credential from the connection string entirely, closing off the credential-leak risk that comes with SQL authentication — the same managed-identity pattern used for Key Vault and Storage access elsewhere in Azure applies directly to connecting to the database itself.

Auditing

- Azure SQL / SQL Server auditing writes a durable log of database events (logins, schema changes, specific statement types) to a storage target, giving you the "who did what, when" record that none of the preventive controls above produce on their own — prevention (RLS, encryption, permissions) and detection (auditing) are complementary, not overlapping, controls.

Common confusion

Dynamic Data Masking and Always Encrypted are frequently confused because both "hide" a column's value, but they defend against completely different threats: masking changes what an *unprivileged application user* sees in query results while the true value sits in plaintext in storage, whereas Always Encrypted keeps the value as ciphertext everywhere outside the client driver — including from a privileged database administrator — which is why masking alone is never an acceptable substitute for encrypting a genuinely sensitive column.

Securing the AI and API surface: model endpoints, GraphQL/REST, and MCP

Once a database exposes model endpoints, GraphQL/REST APIs, or MCP tools, the attack surface extends well past traditional login-and-query access — this section is where DP-800 checks that you extend the same security discipline to that newer surface.

Securing model endpoints with Managed Identity

- When T-SQL calls out to an external model (via `sp_invoke_external_rest_endpoint` or an `AI_GENERATE_EMBEDDINGS`-backed external model), authenticating that outbound call with the database's system- or user-assigned managed identity avoids embedding an API key as a stored credential that could leak.

- The managed identity still needs an explicit role assignment on the target resource (for example, **Cognitive Services OpenAI User** on the Azure OpenAI resource) before the call succeeds — creating a DATABASE SCOPED CREDENTIAL with `IDENTITY = 'Managed Identity'` wires the database up to use the identity, but doesn't by itself grant that identity any permissions on the far end.

Securing GraphQL, REST, and MCP endpoints

- An endpoint exposed through Data API builder (REST or GraphQL) or through a SQL/Fabric MCP server is a new perimeter around the database — each entity's configured permissions determine which roles can read, write, or execute through that endpoint, independent of the underlying table's own object-level GRANTS.
- MCP-specific risk is less about network exposure and more about *scope creep*: a tool that's supposed to run read-only reporting queries but is connected with a broadly-privileged account can be induced (by a malicious prompt, a compromised client, or an over-eager agent) into running writes it was never intended to perform — scoping the credential behind an MCP or API endpoint to the minimum it needs is the primary defense.
- Rate limiting, authentication requirements, and audit logging apply to these endpoints the same way they would to any other externally reachable API — treating a GraphQL or MCP endpoint as "just internal tooling" and skipping those controls is a common, exam-relevant mistake.

Common confusion

Enabling Microsoft Entra-based (passwordless) authentication for direct database connections and securing a GraphQL/REST/MCP endpoint in front of that same database are two different perimeters — locking down the direct SQL connection doesn't automatically constrain what an API or MCP layer sitting in front of it is configured to allow, since that layer typically connects to the database with its own service identity and enforces (or fails to enforce) its own separate permission model.

Performance: isolation levels, execution plans, DMVs, and Query Store

Performance troubleshooting on DP-800 follows a consistent pattern: understand the concurrency model that's causing contention, then use the engine's built-in diagnostic surface to find exactly which query or plan is responsible.

Transaction isolation levels and concurrency

- `READ UNCOMMITTED` allows dirty reads (seeing another transaction's uncommitted changes) in exchange for the least blocking; `READ COMMITTED` (the default) never reads uncommitted data but can still see different results for the same query re-run within one transaction; `REPEATABLE READ` and `SERIALIZABLE` progressively lock more to guarantee re-reads stay stable, at the cost of more blocking.
- `READ COMMITTED SNAPSHOT` and `SNAPSHOT` isolation use row versioning instead of locks for read consistency — readers don't block writers and writers don't block readers, which is why they're often reached for specifically to reduce blocking without loosening isolation guarantees the way `READ UNCOMMITTED` does.
- Choosing an isolation level is a genuine trade-off between data consistency and concurrency throughput, not a purely technical default to leave alone — a reporting workload might deliberately accept `READ UNCOMMITTED`'s looser guarantees for less blocking against an OLTP workload running concurrently.

Diagnosing blocking and deadlocks

- Blocking is one session waiting for a lock another session holds — normal and often transient, but a problem when a long-running transaction holds locks far longer than necessary. `sys.dm_exec_requests` and `sys.dm_os_waiting_tasks` surface who's blocked and who's holding the lock they're waiting on.

- A deadlock is a cycle of blocking (session A waits on B, B waits on A) with no possible resolution, which SQL Server detects automatically and resolves by killing one session as the "deadlock victim" — the exam expects you to know this is automatic, not something an operator manually resolves in real time, though the victim's application still needs retry logic to recover gracefully.

Execution plans and DMVs

- An execution plan shows the operators (scans, seeks, joins, sorts) the optimizer chose and their relative cost — a table scan where a seek was expected, or a sort/hash operation consuming disproportionate resources, is usually the first thing to investigate in a slow query.
- Dynamic management views (DMVs) like `sys.dm_exec_query_stats`, `sys.dm_db_index_usage_stats`, and `sys.dm_os_wait_stats` expose the engine's own internal counters — aggregate wait statistics in particular are often the fastest way to identify *what kind* of bottleneck a workload is hitting (locking, I/O, CPU, memory) before drilling into any one query.

Query Store and Query Performance Insight

- Query Store persists the history of query plans and their runtime statistics over time, which is what makes it possible to answer "did this query get slower after last Tuesday's deployment" — a live DMV snapshot alone can't answer that, since it only reflects the current moment.
- Query Store's plan-forcing capability pins a query to a specific historical plan, directly addressing plan regression (the optimizer picking a new, worse plan after a statistics update or schema change) without changing the query text itself.
- Query Performance Insight (Azure SQL Database) is a portal-level view built on the same underlying data, surfacing top resource-consuming queries over a chosen window without writing a single KQL or T-SQL diagnostic query by hand — the friendlier, less flexible sibling of querying Query Store's catalog views directly.

Common confusion

Blocking and deadlocking are often used interchangeably but aren't the same failure mode: blocking is a normal, usually self-resolving wait for a lock to be released, while a deadlock is a genuine cycle with no resolution path that the engine must break by killing a session — a long blocking chain that never deadlocks can still be a serious performance problem worth investigating, even though nothing ever gets automatically killed the way it does in a true deadlock.

CI/CD with SQL Database Projects

Treating a database schema as versioned, buildable code — rather than a set of manual changes applied directly to production — is the practice this section tests, using SQL Database Projects as the concrete tool.

Projects, DACPACs, and SDK-style projects

- A SQL Database Project (a `.sqlproj`) is a source-controlled representation of a database's schema as individual object files, built into a `.dacpac` (data-tier application package) the same way application code compiles into a binary — the dacpac, not the loose SQL files, is what actually gets deployed.
- The newer SDK-style project format (built on `Microsoft.Build.Sql`) is leaner and more MSBuild-native than the classic SSDT project format, supporting a graphical table designer and richer tooling in Visual Studio while still producing an equivalent dacpac — classic and SDK-style projects both remain supported, with tool support varying slightly across Visual Studio, VS Code, and SSMS.

- `sqlpackage`, the cross-platform CLI, is what actually performs the build-and-deploy actions in automation: `sqlpackage /Action:Extract` reverse-engineers an existing database into project-like files, `/Action:Publish` deploys a dacpac to a target database, and `/Action:Script` generates the deployment T-SQL without running it — the last option being how a pipeline produces a reviewable script instead of auto-applying changes blind.

Schema-drift detection with schema comparison

- Schema compare evaluates the difference between any two of: a connected database, a project, or a dacpac — surfacing exactly which objects would be added, changed, or dropped to make the target match the source, which is the direct tool for catching drift (a change some developer applied straight to a database, bypassing source control).
- Running compare with the project as the *source* and a live database as the *target* is how you validate a database hasn't silently diverged from what's checked in, before that drift causes a confusing failure during the next real deployment.

Reference/static data and pre/post-deployment scripts

- Reference or seed data (lookup tables, configuration rows) that needs to exist identically across every environment is checked into the project as a pre- or post-deployment script — a plain `.sql` file that runs before or after the schema deployment itself, commonly using `MERGE` so it's safe to re-run without duplicating rows.
- SQLCMD variables parameterize environment-specific values (a schema name, a feature flag) inside these scripts and the project's build, so the same project artifact deploys correctly to dev, test, and production without hand-editing anything per environment.

Testing, secrets, and branching

- Unit tests validate individual database objects' logic (a stored procedure's output for known inputs); integration tests validate that a set of objects work correctly together — both run as part of CI before a dacpac is trusted enough to publish anywhere.
- Trunk-based development with short-lived feature branches, pull-request review, and required passing CI checks before merge is the same discipline used for application code, applied to a database project — branching policies, required reviewers ("code owners"), and approval gates on the deployment pipeline itself are what prevent an unreviewed schema change from reaching production.
- Secrets (connection strings, credentials used by the pipeline to reach a target database) are stored in the pipeline's secret store, never hard-coded into the project or a checked-in publish profile — the same principle as keeping API keys out of application source.

Common confusion

`sqlpackage /Action:Publish` and `/Action:Script` both compute the identical set of schema differences under the hood, but only `Publish` actually applies them — `Script` produces the T-SQL that *would* run, which is why a cautious deployment pipeline generates the script first for human review or an approval gate, and only invokes `Publish` (or runs the reviewed script) once that approval is granted, rather than letting every merge auto-publish straight to a shared database.

Integrating with Azure services: Data API builder and change-handling patterns

A database rarely stands alone in production — this section covers turning it into an application-facing API surface with Data API builder, and wiring it into the broader event and observability ecosystem around it.

Data API builder configuration

- Data API builder (DAB) generates both REST and GraphQL endpoints from a single `dab-config.json` describing a data source (connection string, database type) and one or more *entities*, each mapped to a table, view, or stored procedure.
- `dab init --database-type mssql --connection-string "..."` scaffolds the base configuration, and `dab add <EntityName> --source dbo.TableName --permissions "anonymous:*"` registers an entity and its access rules — the resulting `dab-config.json` can also be hand-authored or hand-edited directly rather than only through the CLI.
- Connection strings belong in configuration via the `@env( 'VAR_NAME' )` function rather than as literal text in `dab-config.json` — resolved from an environment variable at runtime, which keeps credentials out of a file that's typically checked into source control alongside the rest of the configuration.
- Per-entity settings control caching (`--cache.enabled`, `--cache.ttl-seconds`, cache level L1 in-process or L1L2 with a distributed second tier), field-level inclusion/exclusion, and — for stored-procedure entities — parameter defaults and requiredness, giving fine control over exactly what each entity exposes without writing any API code by hand.
- Exposing a stored procedure or a view (including one that models a GraphQL relationship across multiple tables) through DAB works the same as exposing a table entity, just with a different `--source.type`.

Deploying DAB

- DAB ships as a container image, so it deploys wherever containers run: Azure Container Apps, Azure App Service, Azure Container Instances, or a self-hosted Docker environment — the deployment target is a hosting decision independent of the configuration file itself, which stays identical across targets.
- A production deployment mounts or bakes in `dab-config.json` (with its connection string resolved via `@env( )`) rather than hard-coding it into the container image, keeping the same image usable across environments by only changing environment variables.

Change-handling patterns

- **Change Data Capture (CDC)** records inserts/updates/deletes from the transaction log into change tables that downstream consumers can poll — built for capturing a full stream of row-level changes for ETL or auditing without adding triggers to the source tables.
- **Change Tracking** is lighter-weight than CDC: it tracks *which* rows changed (and a version number) but not the historical values or every intermediate change, suited to sync scenarios that only need "what's changed since I last checked," not a full change history.
- **Change Event Streaming (CES)** publishes row-level changes as events to a streaming destination (like Event Hubs), letting other services react to a change in near-real time rather than polling change tables on an interval — the event-driven counterpart to CDC's log-based capture.
- **Azure Functions SQL trigger binding** invokes a function automatically when a watched table changes, and **Azure Logic Apps** connectors similarly react to database changes to drive a low-code workflow — both are consumer-side ways to act on a change, layered on top of one of the change-capture mechanisms above.

Observability: Azure Monitor, Application Insights, and Log Analytics

- Azure Monitor collects platform-level metrics and diagnostic logs from the database and any surrounding Azure resources; Application Insights adds distributed tracing and request telemetry when an application (including a DAB-hosted API) is instrumented; Log Analytics is the query layer (KQL) over both, used for everything from an ad hoc investigation to a saved alert rule.
- Wiring diagnostic settings on the database (and on the DAB hosting resource) to send data to a Log Analytics workspace is the setup step that has to happen *before* any of this telemetry exists to query — a common gap that leaves an incident with no historical data to investigate.

Common confusion

CDC and Change Tracking are both change-capture features and are easy to reach for interchangeably, but they answer different questions: Change Tracking tells you *that* a row changed (and to what version), fitting a sync scenario that only needs the current state, while CDC tells you the actual historical sequence of values a row went through, which is what a downstream system needs if it has to replay or audit every individual change rather than just catch up to the latest state.

CHAPTER 3

Implement AI Capabilities in Database Solutions

Register external models and generate embeddings in T-SQL, build exact and approximate vector search with the native VECTOR type, and assemble retrieval-augmented generation with `sp_invoke_external_rest_endpoint`.

External models and embeddings: CREATE EXTERNAL MODEL and chunking

DP-800's AI domain is built around a simple idea the exam tests repeatedly: the SQL Database Engine can call out to AI models and generate embeddings natively in T-SQL, without an external orchestration layer.

Registering an external model

- `CREATE EXTERNAL MODEL model_name WITH (LOCATION = '...', API_FORMAT = 'Azure OpenAI', MODEL_TYPE = EMBEDDINGS, MODEL = 'text-embedding-3-small', CREDENTIAL = [...])` registers a callable AI endpoint as a first-class database object — `API_FORMAT` also accepts `OpenAI`, `Ollama`, and `ONNX Runtime`, covering both cloud-hosted and locally-run models.
- Authentication to the endpoint is handled by a `DATABASE SCOPED CREDENTIAL`, created with `IDENTITY = 'Managed Identity'` (no stored secret, the production-preferred pattern), `'HTTPEndpointHeaders'` (an API key sent as a header), or a couple of other identity types shared with `sp_invoke_external_rest_endpoint` — the credential name is then passed as `CREATE EXTERNAL MODEL`'s `CREDENTIAL` argument.
- `PARAMETERS` accepts a JSON string of runtime parameters appended to every request the model makes (for example `'{"dimensions": 1536}'` to fix an embedding's dimensionality, or a `sql_rest_options.retry_count` for automatic retry on transient failures) — these can also be overridden per call rather than fixed at registration time.
- Using an external model requires `EXECUTE` permission granted explicitly (`GRANT EXECUTE ON EXTERNAL MODEL::model_name TO [<principal>]`), separate from the broader `CREATE EXTERNAL MODEL/ALTER ANY EXTERNAL MODEL` permission needed to register or change one in the first place.

Generating embeddings inline

- `AI_GENERATE_EMBEDDINGS(source USE MODEL model_identifier [PARAMETERS json])` generates an embedding directly inside an ordinary `SELECT`, `INSERT`, or `UPDATE` statement — no separate application call or pipeline step is needed to turn text into a vector.
- A typical enrichment pattern is a single `UPDATE ... SET embedding_col = AI_GENERATE_EMBEDDINGS(text_col USE MODEL MyModel) FROM table`, and this requires the external rest endpoint enabled server configuration to be turned on (`EXECUTE sp_configure 'external rest endpoint enabled', 1; RECONFIGURE WITH OVERRIDE;`) — already enabled by default on Azure SQL Database and SQL database in Fabric, but off by default on SQL Server 2025 and Azure SQL Managed Instance until explicitly configured.

Chunking design

- `AI_GENERATE_CHUNKS(SOURCE = column_or_expression, CHUNK_TYPE = FIXED, CHUNK_SIZE = 100)` splits long text into fragments sized to fit an embedding model's context window, used with `CROSS APPLY` so each source row fans out into one row per chunk: `... CROSS APPLY AI_GENERATE_CHUNKS(SOURCE = d.content, CHUNK_TYPE = FIXED, CHUNK_SIZE = 100) AS c`.
- Chunk size is a genuine design trade-off, not just a technical limit to work around: smaller chunks give more precise retrieval (a matched chunk is more likely to be narrowly relevant) at the cost of losing surrounding context, while larger chunks preserve context at the cost of diluting a chunk's semantic focus — the right size depends on the source content's structure and how the chunks will be consumed downstream.
- Choosing *which columns* to embed matters as much as chunk size: embedding a free-text description column serves semantic search well, but embedding a rigid, low-cardinality column (a status code) wastes storage and compute without improving retrieval quality, since exact/relational filtering already handles that column better than vector similarity would.

Keeping embeddings fresh

- An embedding generated once goes stale the moment its source text changes, so a maintenance strategy has to re-run generation on updates — options mirror the change-handling mechanisms covered in the previous domain: a table trigger that recomputes the embedding synchronously on write, or an asynchronous pattern driven by Change Tracking, CDC, an Azure Functions SQL trigger binding, Azure Logic Apps, Change Event Streaming, or a scheduled Microsoft Foundry pipeline.
- The synchronous-trigger approach keeps embeddings always current at the cost of adding embedding-generation latency to every write; the asynchronous approaches decouple that latency from the write path at the cost of a window where the embedding is briefly stale relative to its source row — which trade-off is acceptable depends entirely on how quickly a change needs to be searchable.

Common confusion

`CREATE EXTERNAL MODEL` and `AI_GENERATE_EMBEDDINGS` are two separate steps that are easy to conflate: `CREATE EXTERNAL MODEL` is a one-time registration of *where* the model lives and *how* to authenticate to it, while `AI_GENERATE_EMBEDDINGS` is the function called per-query or per-row that actually invokes it — you can't call `AI_GENERATE_EMBEDDINGS` against a model that hasn't been registered first, and registering a model does nothing on its own until something calls it.

The vector data type and exact nearest-neighbor search

Native vector support is the foundational AI feature the rest of this domain builds on — the exam expects precise recall of the type's syntax and limits, not just conceptual familiarity with "storing embeddings."

The VECTOR data type

- Column syntax is `column_name VECTOR(dimensions [, base_type])` — the default base type is `float32`; specifying `float16` explicitly stores each element at half precision, trading some numeric fidelity for roughly half the storage footprint.
- A vector must have at least one dimension, and the maximum supported is **1998** dimensions — a model that emits a higher-dimensional embedding (some larger embedding models exceed this) has to be dimensionality-reduced or truncated before it fits a native `VECTOR` column.
- Vectors are stored in an optimized binary format internally but exposed as JSON arrays for convenience — casting a JSON array literal to `VECTOR(n)` (`CAST('[1.0, -0.2, 30]' AS VECTOR(3))`), or the implicit form `DECLARE @v VECTOR(3) = '[1.0, -0.2, 30]'` is the standard way to construct one, and casting a vector back to `NVARCHAR(MAX)` or JSON reverses it.

- VECTOR can be used as a table column, a variable, and a stored procedure or function parameter (including as an OUTPUT parameter) — it behaves like any other SQL Server data type in those respects.

Exact search with VECTOR_DISTANCE

- VECTOR_DISTANCE(distance_metric, vector1, vector2) computes the distance between two vectors using 'cosine', 'euclidean', or 'dot' (negative dot product) as the metric — it is always exact, and critically, it never uses a vector index even if one exists on the column.
- A typical k-nearest-neighbor (kNN) query pattern is `SELECT TOP (10) id, title, VECTOR_DISTANCE('cosine', @queryVector, content_vector) AS distance FROM table ORDER BY distance` — because this is a brute-force scan across every candidate row, Microsoft's general guidance is to use exact search when the searchable set is under roughly 50,000 vectors (after any WHERE-clause filtering has already narrowed the candidate set), and to reach for approximate search covered next once it's larger.
- A query can also filter directly on a distance threshold (`WHERE VECTOR_DISTANCE('cosine', @v, title_vector) < 0.3`) rather than only taking a fixed TOP N, which is the right shape when "everything reasonably similar" matters more than "the fixed top 10."

Common confusion

VECTOR_DISTANCE will happily run against a column that has a vector index defined on it, and will still return exact results every single time — the function's own documentation is explicit that it never uses a vector index, so seeing an index exist on a column is not evidence that a given query is using approximate search; only VECTOR_SEARCH (next section) can actually use one.

Approximate search with VECTOR_SEARCH, vector indexes, and ANN vs. ENN

Once exact search stops scaling, approximate nearest neighbor (ANN) search — built on a dedicated vector index and the VECTOR_SEARCH function — trades a small amount of recall accuracy for search that stays fast as the dataset grows into the millions of rows.

Exact (ENN) vs. approximate (ANN) search

- Exact nearest neighbor (ENN) search — what VECTOR_DISTANCE-based kNN queries perform — calculates distance against every candidate row and is guaranteed to find the true nearest neighbors, at linear cost in the number of rows.
- Approximate nearest neighbor (ANN) search uses a specialized index structure to avoid scanning every row, returning results that are very likely but not *guaranteed* to be the true nearest neighbors — the trade a workload makes deliberately once dataset size makes exhaustive scanning too slow to be practical.
- Approximate vector index and vector search are preview features, currently available in SQL Server 2025, Azure SQL Database, and SQL database in Microsoft Fabric.

Creating and using a vector index

- `CREATE VECTOR INDEX idx_name ON table(vector_column) WITH (METRIC = 'cosine')` builds the ANN index structure used by approximate search.
- `VECTOR_SEARCH(TABLE = table AS alias, COLUMN = vector_column, SIMILAR_TO = @queryVector, METRIC = 'cosine')` is a table-valued function returning every column from the target table plus a computed distance column — when referencing an alias in the TABLE argument, that same alias (not VECTOR_SEARCH's own result alias) is what you use to reference the table's other columns in the SELECT.

- `SELECT TOP (N) WITH APPROXIMATE ... ORDER BY distance` is what actually triggers approximate (ANN) execution — the `ORDER BY` must reference only the distance column in ascending order, and using `WITH APPROXIMATE` without a `VECTOR_SEARCH` function in the query, or omitting `TOP/ORDER BY` requirements, raises an error rather than silently falling back to exact search.
- Without `WITH APPROXIMATE`, a `VECTOR_SEARCH` query still runs, but as an exact kNN scan — meaning `VECTOR_SEARCH` itself doesn't guarantee approximate execution; the `WITH APPROXIMATE` clause is what does. A newer engine behavior also applies relational `WHERE` predicates *during* the vector search (iterative filtering) rather than only after the fact, which fixed a historical problem where filtering after the fact could return fewer rows than actually existed.
- The `FORCE_ANN_ONLY` table hint forces the optimizer to use the approximate index specifically rather than letting it choose an execution strategy — it requires both an existing vector index and `SELECT TOP (N) WITH APPROXIMATE` already being used; specifying it without either fails.
- Vector-indexed tables cannot be truncated directly with `TRUNCATE TABLE` — the index has to be dropped first, the table truncated and repopulated with at least 100 rows, and the index recreated afterward.

Choosing and evaluating vector index types and metrics

- The distance metric (cosine, euclidean, or dot) has to match how the embedding model itself was trained to be compared — using the wrong metric for a given embedding model silently degrades relevance without producing any error.
- Evaluating a vector or hybrid search setup means measuring recall (did the approximate index actually surface the true nearest neighbors often enough) against the latency and cost savings ANN provides over exhaustive ENN scanning — the acceptable accuracy loss is a business decision informed by testing on representative queries, not a fixed number.

Common confusion

Adding a vector index to a table and using `VECTOR_SEARCH` in a query are each necessary but not sufficient on their own for approximate search to actually happen: a vector index with no `WITH APPROXIMATE` clause in the query still yields an exact kNN scan, and `WITH APPROXIMATE` with no vector index present is simply an error — getting ANN execution requires the index, the `VECTOR_SEARCH` function, and the `TOP (N) WITH APPROXIMATE ... ORDER BY distance` syntax all present together.

Full-text, hybrid search, and Reciprocal Rank Fusion

Vector similarity alone doesn't win every retrieval scenario — this section covers when to reach for keyword-based full-text search instead, and how hybrid search combines both approaches for better relevance than either alone.

Choosing among full-text, vector, and hybrid search

- Full-text search excels at exact terminology — product codes, proper nouns, acronyms, and precise keyword matches — cases where a semantically "close" vector match can actually be the *wrong* answer because it's topically similar but not textually correct.
- Vector (semantic) search excels at conceptual similarity — finding relevant content that's worded completely differently from the query — but can miss an exact identifier or rare term that isn't well represented in the embedding space.
- Hybrid search runs both a full-text (or keyword) query and a vector similarity query against the same request and merges their results, directly compensating for each approach's individual blind spot rather than forcing a single choice between them.

Implementing full-text search

- Full-text search requires a full-text index built on the searchable column(s), queried with predicates like `CONTAINS` or `FREETEXT` rather than a plain `LIKE` — it supports linguistic features (stemming, thesaurus, proximity search) that `LIKE` and even regex functions don't provide, since it's purpose-built for natural-language text search rather than pattern matching.

Merging results with Reciprocal Rank Fusion

- Reciprocal Rank Fusion (RRF) combines two or more ranked result lists (a keyword-search ranking and a vector-search ranking) into one final ranking, scoring each document by the sum of $1 / (k + \text{rank})$ across every list it appears in — a document ranked highly by *either* method contributes a large score, while one that ranks well in both compounds to an even higher combined score.
- RRF is specifically a *rank-based* fusion technique — it only needs each result's position in its respective ranked list, not the raw, differently-scaled similarity or relevance scores each method produces internally, which is exactly what makes it possible to combine two methods (full-text relevance scores and cosine vector distances) that otherwise aren't on a comparable numeric scale at all.

Evaluating vector and hybrid search performance

- Retrieval quality is measured independently of generation quality: precision and recall against a labeled set of "actually relevant" documents, and rank-aware metrics that reward relevant results appearing near the top, apply to a hybrid retriever the same way they would to any search-ranking system.
- Because hybrid search has more tunable pieces than either approach alone (the full-text query shape, the vector similarity threshold, and how RRF weighs the two), isolating what caused a relevance improvement or regression means changing one variable at a time against a fixed, representative test query set rather than adjusting several knobs simultaneously.

Common confusion

Hybrid search and simply running two separate queries and eyeballing both result sets are not the same thing — the value of hybrid search specifically comes from a principled fusion step like RRF that produces one coherent ranking from both signals, rather than leaving it to the caller (or the end user) to reconcile two independently-ranked, differently-scored lists by hand.

Retrieval-augmented generation with `sp_invoke_external_rest_endpoint`

RAG is the capstone pattern this domain builds toward — pulling relevant data out of the database, handing it to a language model as context, and returning that model's answer, entirely orchestrated from T-SQL.

Identifying RAG use cases

- RAG fits scenarios where an LLM needs to answer questions grounded in specific, current, or proprietary data it wasn't trained on — the model supplies natural-language fluency and reasoning, while the database supplies the facts, retrieved fresh at query time rather than baked into the model's training data.
- Because retrieval happens against live data, a RAG answer reflects data as of right now (a price change, a newly added row) in a way that would otherwise require retraining or fine-tuning a model to keep up with — the trade RAG makes is added query-time latency (the retrieval step, then the model call) in exchange for that freshness.

Converting structured data to JSON for LLM consumption

- A language model consumes text, so relational query results need to be shaped into JSON before being sent as prompt context — `JSON_OBJECT`, `JSON_ARRAY`, and `FOR JSON` all serialize rows and columns into the JSON text a prompt template can embed directly.
- Retrieval typically runs a hybrid or vector search (from the previous section) first to select only the *relevant* rows, then serializes just that narrowed result set to JSON — sending an entire table as context would both blow past the model's context window and dilute the genuinely relevant information with noise.

Prompting via `sp_invoke_external_rest_endpoint`

- `sys.sp_invoke_external_rest_endpoint @url = N'https://...', @payload = N'{...}', @method = N'POST', @headers = N'{...}', @credential = [...], @timeout = 30, @retry_count = 3, @response = @response` OUTPUT is the general-purpose stored procedure for calling any HTTPS REST endpoint from T-SQL — including an Azure OpenAI chat completion endpoint — and it's what actually sends the assembled prompt (system instructions plus the retrieved JSON context plus the user's question) to the model.
- Authentication uses the same `DATABASE SCOPED CREDENTIAL` mechanism as external models: Managed Identity sends the system-assigned identity in request headers, `HTTPEndpointHeaders/HTTPEndpointQueryString` inject a stored secret (like an API key) into headers or the query string, and Shared Access Signature provides delegated, time-limited access — a database user needs `REFERENCES` permission on the credential to use it via `@credential`.
- `sp_invoke_external_rest_endpoint` is disabled by default in SQL Server 2025 and Azure SQL Managed Instance (enabled with `EXECUTE sp_configure 'external rest endpoint enabled', 1; RECONFIGURE WITH OVERRIDE;`) but enabled by default in Azure SQL Database and SQL database in Fabric — and calling it at all requires the `EXECUTE ANY EXTERNAL ENDPOINT` database permission.
- Concurrent outbound calls through the procedure are throttled to roughly 10% of worker threads (capped at 150), enforced at the database level and, for an elastic pool, at the pool level too — a RAG workload issuing many concurrent model calls can hit this throttle well before it hits any model-side rate limit, and `sys.dm_user_db_resource_governance` reports the actual configured limits for a given service tier.

Extracting language-model responses

- The `@response` output parameter receives a JSON (or XML) envelope containing HTTP status/headers under a `response` key and the actual payload the model returned under a `result` key — extracting the model's generated text back out means applying `JSON_VALUE/JSON_QUERY` to that `result` portion of the response, the same JSON functions used everywhere else in this domain.
- Checking the HTTP status embedded in `@response` (or the procedure's own return value, which is 0 on a 2xx response and the HTTP status code otherwise) before trusting the payload matters because a non-2xx response still returns successfully from the procedure's perspective unless the call couldn't be made at all — silently parsing an error response as if it were a valid model answer is an easy mistake in a first RAG implementation.

Common confusion

Generating an embedding (`AI_GENERATE_EMBEDDINGS`, used for the retrieval half of RAG) and generating a natural-language answer (`sp_invoke_external_rest_endpoint` calling a chat/completion model, used for the generation half) are frequently conflated because both are "calling AI from T-SQL," but they're different calls to conceptually different kinds of models — a RAG pipeline needs both steps in sequence, first embedding the query to retrieve relevant rows via vector or hybrid search, then separately sending those retrieved rows plus the original question to a generative model to produce the final answer.

CHAPTER 4

Quick-reference: common confusions

Every “Common confusion” callout from this guide, gathered here as a last-day revision sheet.

Design and Develop Database Solutions

Ledger tables and temporal tables both keep history, but for different reasons: temporal tables exist so *any* authorized user or application can query past states for business logic (an as-of report), while ledger tables exist specifically to make tampering by a privileged user (a DBA editing rows directly) cryptographically detectable — a ledger table can also be temporal under the hood, but the two features solve different trust problems and aren't interchangeable.

A view and an inline table-valued function look similar (both wrap a `SELECT` and both get inlined into the caller's plan), but only a function can take parameters — a view's filtering logic has to live in the caller's `WHERE` clause, while a function can encapsulate parameterized logic (like a security predicate that takes a user ID) that a view structurally can't express on its own.

A window function and a `GROUP BY` aggregate can compute the same underlying math (a sum, a count) but return fundamentally different shapes: `GROUP BY` collapses the result to one row per group, while a window function keeps every original row and attaches the computed value alongside it — needing both a detail row and an aggregate value side by side is exactly when a window function is the right tool and `GROUP BY` alone isn't.

`REGEXP_LIKE` and the classic `LIKE` operator both test a string against a pattern, but `LIKE`'s wildcard vocabulary (`%`, `_`, `[ . . . ]`) is far more limited than full regular expression syntax (alternation, quantifiers, character classes, capture groups) — reaching for `REGEXP_LIKE` only when `LIKE` genuinely can't express the pattern keeps queries both correct and easier to optimize, since a plain `LIKE` with a leading literal can still use an index seek in ways a regex generally can't.

Enabling GitHub Copilot for a SQL project and connecting it to an MCP server are two separate steps that are easy to conflate: Copilot alone can generate T-SQL text based on what's visible in your editor and instruction files, but it can't see live database schema or execute anything until an MCP server is added, its tools are individually enabled, and (in Agent mode) each tool call is approved.

Secure, Optimize, and Deploy Database Solutions

Dynamic Data Masking and Always Encrypted are frequently confused because both “hide” a column's value, but they defend against completely different threats: masking changes what an *unprivileged application user* sees in query results while the true value sits in plaintext in storage, whereas Always Encrypted keeps the value as ciphertext everywhere outside the client driver — including from a privileged database administrator — which is why masking alone is never an acceptable substitute for encrypting a genuinely sensitive column.

Enabling Microsoft Entra–based (passwordless) authentication for direct database connections and securing a GraphQL/REST/MCP endpoint in front of that same database are two different perimeters — locking down the direct SQL connection doesn't automatically constrain what an API or MCP layer sitting in front of it is configured to allow, since that layer typically connects to the database with its own service identity and enforces (or fails to enforce) its own separate permission model.

Blocking and deadlocking are often used interchangeably but aren't the same failure mode: blocking is a normal, usually self-resolving wait for a lock to be released, while a deadlock is a genuine cycle with no resolution path that the engine must break by killing a session — a long blocking chain that never deadlocks

can still be a serious performance problem worth investigating, even though nothing ever gets automatically killed the way it does in a true deadlock.

`sqlpackage /Action:Publish` and `/Action:Script` both compute the identical set of schema differences under the hood, but only `Publish` actually applies them — `Script` produces the T-SQL that *would* run, which is why a cautious deployment pipeline generates the script first for human review or an approval gate, and only invokes `Publish` (or runs the reviewed script) once that approval is granted, rather than letting every merge auto-publish straight to a shared database.

CDC and Change Tracking are both change-capture features and are easy to reach for interchangeably, but they answer different questions: Change Tracking tells you *that* a row changed (and to what version), fitting a sync scenario that only needs the current state, while CDC tells you the actual historical sequence of values a row went through, which is what a downstream system needs if it has to replay or audit every individual change rather than just catch up to the latest state.

Implement AI Capabilities in Database Solutions

`CREATE EXTERNAL MODEL` and `AI_GENERATE_EMBEDDINGS` are two separate steps that are easy to conflate: `CREATE EXTERNAL MODEL` is a one-time registration of *where* the model lives and *how* to authenticate to it, while `AI_GENERATE_EMBEDDINGS` is the function called per-query or per-row that actually invokes it — you can't call `AI_GENERATE_EMBEDDINGS` against a model that hasn't been registered first, and registering a model does nothing on its own until something calls it.

`VECTOR_DISTANCE` will happily run against a column that has a vector index defined on it, and will still return exact results every single time — the function's own documentation is explicit that it never uses a vector index, so seeing an index exist on a column is not evidence that a given query is using approximate search; only `VECTOR_SEARCH` (next section) can actually use one.

Adding a vector index to a table and using `VECTOR_SEARCH` in a query are each necessary but not sufficient on their own for approximate search to actually happen: a vector index with no `WITH APPROXIMATE` clause in the query still yields an exact kNN scan, and `WITH APPROXIMATE` with no vector index present is simply an error — getting ANN execution requires the index, the `VECTOR_SEARCH` function, and the `TOP (N) WITH APPROXIMATE ... ORDER BY distance` syntax all present together.

Hybrid search and simply running two separate queries and eyeballing both result sets are not the same thing — the value of hybrid search specifically comes from a principled fusion step like RRF that produces one coherent ranking from both signals, rather than leaving it to the caller (or the end user) to reconcile two independently-ranked, differently-scored lists by hand.

Generating an embedding (`AI_GENERATE_EMBEDDINGS`, used for the retrieval half of RAG) and generating a natural-language answer (`sp_invoke_external_rest_endpoint` calling a chat/completion model, used for the generation half) are frequently conflated because both are "calling AI from T-SQL," but they're different calls to conceptually different kinds of models — a RAG pipeline needs both steps in sequence, first embedding the query to retrieve relevant rows via vector or hybrid search, then separately sending those retrieved rows plus the original question to a generative model to produce the final answer.